Circuit Coach: Technical Design

Team: Impact Theory | **Problem 3B:** Identifying what a learner needs next
Prototype: https://dfsi.vercel.app/ | **Code:** `backend/` and `frontend/` in the project repository

1. Introduction

Our project, **Problem 3B: Identifying what a learner needs next**, works with Barabari Collective and Nirmaan, which run skilling and vocational programmes. Learners there study in short sessions, often on their phones, and learning happens across classes, videos and practice. Because of this, a learner may not realise they have misunderstood a concept until much later. Trainers, who look after large batches, cannot always tell who is struggling or with what.

Circuit Coach is our response. After each short quiz, it works out which concept a learner is finding difficult, suggests one short lesson to review it, and brings in the trainer if the learner is still stuck. We use a basic electrician course as the example.

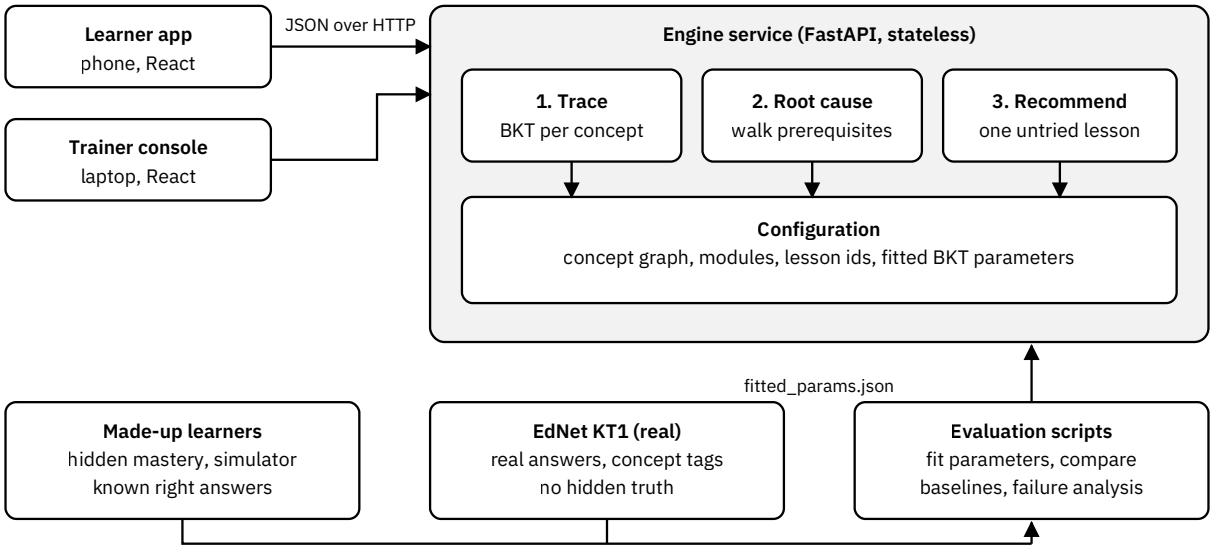
In Sprint 1 we designed the product and built a clickable prototype of the learner app and the trainer console. The prototype showed what the product should do, but its "intelligence" was a simple stand-in running in the browser. This document covers Sprint 2: the real technical core, which is the engine that estimates what each learner understands and decides what they should do next, and how well it performs.

2. Purpose

Sprint 1 defined the product and its UI. Sprint 2 builds the hardest technical part first: the **knowledge-tracing and recommendation engine**, and measures a **baseline** for it. The baseline is standard Bayesian Knowledge Tracing (BKT), one model per concept, with a simple walk back through prerequisites. We kept it deliberately plain so that Sprint 3 improvements can be measured against it.

3. Technical specification

Inputs: a learner's answers (concept tags, correct or not, "I'm not sure"), the concept graph, and the lessons available per concept. **Outputs:** a mastery probability per concept, and one recommendation: `{concept_id, diagnostic_text, recommended_content_id}`, as in the Sprint 1 design doc.



Components. Dataset and evaluation code is offline; only the engine service runs when learners use the app.

Design decisions

- **Stateless service.** Each request carries the learner's state (`mastery`, `observations`, `tried`) and returns the updated state. The engine is easy to test and to call from the prototype. Storing learners is a separate piece of work.

- **One model per concept.** Each concept is a two-state model (not mastered, mastered) with four parameters: starting mastery, chance of learning after each answer, slip (wrong while mastered) and guess (right while not mastered).
- **Multi-concept questions.** A question tagged with several concepts updates each of them using the same answer (naive credit assignment).
- **"I'm not sure".** Treated as a wrong answer with almost no chance of a lucky guess.
- **Gap threshold.** A concept below 0.6 is a gap and 0.8 or above is strong. These are the thresholds the UI uses for words like Needs practice.
- **Root cause.** From the weakest concept just assessed, walk back to the weakest prerequisite that is also a gap and has been asked about, and recommend that.
- **One lesson.** The first lesson for that concept that the learner has not already tried.
- **Fitting.** Parameters are fitted per concept by expectation maximisation (EM) with sensible bounds. A unit test checks that EM recovers known parameters from generated data.

Endpoint	Purpose
`POST /v1/onboarding`	Seed a new learner from experience and the placement quiz, return the start module
`POST /v1/quiz-result`	Update the model from a quick check and return the recommendation. A failed follow-up check returns a repeat-gap recommendation with a new lesson
`POST /v1/got-it`	Apply a learning step after "Got it!"
`GET /v1/concepts`, `GET /health`	Concept graph and lessons, service status

4. Initial implementation

- **Engine service:** Python and FastAPI, in `backend/circuit_coach/`. It is a port of the engine already running in the prototype, so behaviour matches.
- **Prototype connection:** with `VITE_API_URL` set, the placement, quiz-result and "Got it!" steps go to the service, and the top bar shows "Engine: API". If the service is unreachable the prototype falls back to its in-browser engine and says so. We tested the full learner story (placement, diagram gap, "Got it!") against the live service, and the fallback with the service stopped.
- **Tests:** 29 automated tests cover the BKT maths, EM recovery, the engine (direct gap, root cause, repeat gap, tried lessons), the API, the simulator, and a check that the Python concept graph still matches the frontend's.
- **Evaluation code:** a made-up learner generator, a ladder of predictors, the diagnosis evaluation, an EdNet loader, and three analysis scripts. All are reproducible from the commands in `backend/README.md`.

5. Evaluation design

Made-up learners. Real data from the NGOs does not exist yet, and real datasets do not say which concept a learner truly lacked. So we generate learners with a hidden mastered or not-mastered state per concept. The assumptions are written at the top of `simulate.py`. In short:

- Starting mastery depends on experience and on prerequisites.
- Every answer is a chance to learn, but learning is five times slower while a prerequisite is not mastered. This is exactly what plain BKT cannot see.
- A question tagged with several concepts needs all of them.
- Learners who do not know sometimes choose "I'm not sure".

The result is 3,000 learners per run with 5 runs (70% to fit, 30% to test), about 30 answers each (88,571 answers in a run).

Real data. EdNet KT1 (Choi et al., 2020): real answers from a Korean English-test tutoring app, under a research licence (CC BY-NC 4.0). We used a random sample of 14,460 students with at least 30 answers (357,756 test answers) and its 189 concept tags. It has no prerequisite graph and no hidden truth, so it can only test **next-answer prediction**.

Measures. Next-answer prediction uses AUC, accuracy, log loss and RMSE. For diagnosis, at the end of each module quick check we compare the engine with the hidden truth:

- **True gap:** an assessed concept the learner has not mastered.
- **True blocker:** an unmastered concept with all prerequisites mastered. It is what must be fixed first.
- **Hidden-prerequisite case:** the real blocker is upstream of the assessed gap.

Baselines. Global average; each question's own accuracy; the learner's running accuracy on the tagged concepts; BKT with hand-set parameters; BKT with fitted parameters. For diagnosis, the status quo is "study the concept you got wrong".

6. Baseline results

6.1 Made-up learners: predicting the next answer

Method	AUC	Accuracy	Log loss
Global average	0.500	0.652	0.648
Per-question accuracy	0.603	0.665	0.628
Learner's running accuracy	0.613	0.681	0.631
BKT, hand-set parameters	0.649	0.713	0.598
BKT, fitted	**0.688**	0.701	**0.595**

Run-to-run spread is at most 0.01. BKT is the best predictor, and fitting parameters raises AUC by 0.04. Accuracy is slightly lower, so we treat the two BKT versions as close.

6.2 Made-up learners: finding the gap and the lesson (fitted parameters)

Method	Gap F1	Recommendation is a true blocker	Hidden-prerequisite cases solved	Missed gaps	False alarms
Study what you got wrong	87.0%	31.2%	7.4%	1.0%	50.0%
BKT, assessed concept only	89.1%	28.5%	5.0%	2.6%	31.9%
BKT + prerequisite walk	**89.1%**	**59.2%**	**61.9%**	2.6%	31.9%

- BKT finds gaps slightly better than the status quo and cuts false alarms (a lesson recommended when the learner has no gap) from 50% to 32%.
- **The prerequisite walk is what finds hidden causes.** Without it, the engine almost never recommends the real blocker (5.0% of hidden cases); with it, 61.9%.
- Mastery estimates match the hidden truth with AUC 0.85.

6.3 Real data: EdNet

Method	AUC	Accuracy	Log loss
Global average	0.500	0.619	0.665
Per-question accuracy	**0.721**	**0.688**	**0.590**
Learner's running accuracy	0.598	0.615	0.657
BKT, hand-set parameters	0.570	0.517	0.767
BKT, fitted, all tags	0.627	0.625	0.645
BKT, fitted, first tag only	0.641	0.647	0.633

Fitted BKT beats the learner's own running accuracy and the hand-set version, but **loses clearly to simple per-question accuracy.**

7. What worked and what did not

Worked

- Standard BKT is a sound baseline. It beats simple baselines on made-up learners, and fitting helps on both datasets.
- The prerequisite walk turns a model that almost never finds hidden causes into one that does so about 62% of the time.
- The service matches the prototype's behaviour, and the prototype degrades safely when it is down.

Did not work, with evidence

1. **Multi-concept questions distort the model.**
   - Fitting on all tagged answers puts the mean slip at 0.25. Fitting on single-concept questions only gives 0.14, against a true value near 0.09. Five of ten concepts hit the 0.30 limit, against one of ten.
   - On EdNet, using only the first tag beats using all tags (AUC 0.641 against 0.627).
   - In the made-up data, **31.6% of hidden-prerequisite misses recommend a concept the learner actually knew.** A wrong answer on a two-concept question blames both.
2. **BKT ignores question difficulty.** On real data it trails per-question accuracy by 0.08 AUC, because real questions differ a lot in difficulty.
3. **Too many false alarms.** About 32% of checks with no true gap still trigger a lesson, because one or two answers per concept is little evidence.
4. **BKT does not know prerequisites in the model itself.** The walk is a rule added afterwards. It cannot use the fact that learning is slower while a prerequisite is missing.

Failure analysis: hidden-prerequisite cases	Share
Found the true blocker	60.5%
Recommended a concept the learner actually knew	31.6%
The true blocker was asked about but looked fine	4.1%
Recommended a real gap, but the real blocker was further upstream	2.5%
Engine saw no gap at all	1.3%